AICTE Approval & Compliance Orchestration Platform

Problem Statement 26262 — Testing Guide for Backend-Connected UI Features

Target Stack: Spring Boot 3.4.3 + PostgreSQL 18.6 + React 19 **Document:** v1.1.0 **Date:** October 2026

AICTE Approval & Compliance Orchestration Platform

Target Problem Statement: 26262

Target Environment: Spring Boot 3.4.3 + PostgreSQL 18.6 + React 19 Frontend

Audience: Technical Evaluators, Manual Testers, and Student Demonstrators

Document Version: 1.0.0 (Production-Verified)

Date: October 2026

Executive Summary & Scope

This testing guide provides step-by-step procedures for manually testing and verifying **only the UI features that communicate with the Spring Boot backend and persist or retrieve data through PostgreSQL**.

1. Scope Matrix

UI Feature / Screen	Backend Endpoint	PostgreSQL Persistence	Scope Status
Demo Login & Role Switcher	`POST /api/v1/auth/token` `GET /api/v1/auth/me`	Memory Dev Token Store (Validates Server Roles)	**IN SCOPE** (Section 4, TC-01)
Applications Explorer List	`GET /api/v1/applications?size=100`	`applications` JOIN `institutions`	**IN SCOPE** (Section 5, TC-02)
Application Dossier View	`GET /api/v1/applications/{id}`	`applications`, `workflow_transitions`	**IN SCOPE** (Section 6, TC-03)
Return for Correction Action	`POST /api/v1/applications/{id}/actions`	`applications`, `workflow_transitions`, `audit_events`	**IN SCOPE** (Section 7, TC-04)
Submit Corrections Action	`POST /api/v1/applications/{id}/actions`	`applications`, `workflow_transitions`, `audit_events`	**IN SCOPE** (Section 8, TC-05)
Complete Scrutiny Action	`POST /api/v1/applications/{id}/actions`	`applications`, `workflow_transitions`, `audit_events`	**IN SCOPE** (Section 9, TC-06)
Invalid State Transition Guard	`POST /api/v1/applications/{id}/actions`	Rejection (RFC 7807 400 Problem Detail)	**IN SCOPE** (Section 10, TC-07)
End-to-End Scrutiny Journey	Full Journey Flow	Full Audit & Transition Persistence	**IN SCOPE** (Section 11, TC-08)
Create Application Workflow	`POST /api/v1/applications`	`applications`, `audit_events`	**IN SCOPE** (Section 11b, TC-09)
Institution Management CRUD	`POST /api/v1/institutions` (Backend API Only)	None in UI (API exists, no UI screen connected)	**EXCLUDED** (Section 12.2)
EVC Scheduling Module	None (Uses `localStorage`)	None (`aicte_evc_store` in browser)	**OUT OF SCOPE** (Section 13)
Compliance Obligations Module	None (Uses `localStorage`)	None (`aicte_compliance_store` in browser)	**OUT OF SCOPE** (Section 13)
Document Registry & Versions	None (Uses `localStorage`)	None (`aicte_documents_store` in browser)	**OUT OF SCOPE** (Section 13)
System Alerts & Notifications	None (Uses `localStorage`)	None (`aicte_alerts_store` in browser)	**OUT OF SCOPE** (Section 13)

Reports & BI Analytics	None (Calculated in-memory)	None (Local calculation engine)	**OUT OF SCOPE** (Section 13)

---

1. Prerequisites and Environment Startup

Before executing any manual tests, start the database, backend, and frontend services in order.

Step 1.1: Start PostgreSQL via Docker Compose

In your terminal, navigate to the backend repository:

```
cd /home/shreyash/Projects/SIH2
sudo docker compose up -d
```

Verify that PostgreSQL is healthy and accepting connections on port `5432`:

```
pg_isready -h localhost -p 5432
# Expected output: localhost:5432 - accepting connections
```

Step 1.2: Start Spring Boot Backend

Start the application using the Maven Wrapper:

```
cd /home/shreyash/Projects/SIH2
./mvnw spring-boot:run
```

Wait until the startup banner appears and the console shows:

```
Started AictePlatformApplication in X.XXX seconds
```

Verify backend health in a separate terminal:

```
curl -s http://localhost:8080/api/v1/health
# Expected output: {"status":"UP","application":"aicte-compliance-platform",...}
```

Step 1.3: Start React 19 Frontend

Open a third terminal window:

```
cd /home/shreyash/Downloads/sih26262_front
npm run dev
```

The terminal will display:

```
  VITE v6.2.0  ready in XXX ms
  ➜  Local:   http://localhost:5173/
```

Open **Google Chrome** or **Chromium** and browse to `http://localhost:5173`.

Step 1.4: Open Browser Developer Tools

1. Press `F12` (or `Ctrl + Shift + I` / `Cmd + Option + I`).

2. Switch to the **Network** tab.

3. Check the **Fetch/XHR** filter to isolate API calls.

4. Check **Preserve log** so requests are not cleared across role changes.

---

2. Architecture & Data Flow



---

3. Database Verification Reference (Safe Read-Only SQL)

All database verification queries are non-destructive and can be executed anytime using `psql` or DBeaver:

Connection Details

- **Host:** `localhost` | **Port:** `5432`
- **Database:** `aicte_platform_db`
- **Username:** `aicte_app_user`
- **Password:** `aicte_123`

Command-line Quick Query

```
PGPASSWORD=aicte_123 psql -h localhost -U aicte_app_user -d aicte_platform_db -c "
SELECT id, application_reference, current_stage, status, owner_name, version
FROM applications
WHERE application_reference = 'APP-2026-1042';
"
```

---

4. Test Case 1: Demo Login & Role Switching

Test ID: TC-01

- **Feature Name:** Demo Login and Role Authentication Bridge
- **Purpose:** Verify that selecting a demo role contacts the Spring Boot auth endpoint, receives a server-generated Bearer token, stores it in browser memory, and authenticates subsequent API requests.
- **Preconditions:** Backend running on `http://localhost:8080`; Frontend running on `http://localhost:5173`.
- **Primary Endpoint:** `POST /api/v1/auth/token`
- **Validation Endpoint:** `GET /api/v1/auth/me`

Step-by-Step Instructions

1. Open `http://localhost:5173` in your browser.

2. If already logged in, look at the top-right header for the **Role Switcher Dropdown** (showing current user and role badge).

3. Click the dropdown and select **Scrutiny Officer** (`B. Singh`).

4. Observe the Network tab in Developer Tools:

- Locate the request to `/api/v1/auth/token`.

- Confirm Method is `POST`.

- Confirm Request Payload:

`json

{"role": "Scrutiny Officer"}

`

- Confirm Response Status is `200 OK`.

- Confirm Response Body contains an `access_token` and `user` object:

`json

{

"access_token": "dev-token-USR-SO-01-xxxxxx",

"token_type": "bearer",

"user": {

"id": "USR-SO-01",

"name": "B. Singh",

"role": "Scrutiny Officer",

"region": "Northern Region"

}

}

`

5. In Developer Tools, open the **Application** (or **Storage**) tab -> **Local Storage** -> `http://localhost:5173`.

- Verify key `aicte_auth_token` has the value `dev-token-USR-SO-01-xxxxxx`.

- Verify key `aicte_user` contains the serialized user profile.

6. Now switch role to **Institution Representative** (`Dr. Ramesh`).

- Confirm a new `POST /api/v1/auth/token` request is sent with `{"role": "Institution Representative"}`.

- Confirm response returns `access_token` starting with `dev-token-USR-INST-01-`.

Expected Results

- Header immediately reflects the selected user name and role badge.
- `Authorization: Bearer dev-token-...` is stored and sent with all future API requests.

Development Security Limitation Notice

> **Important Limitation:**

> This authentication bridge is strictly designed for local prototype demonstration. It allows evaluators to simulate 7 regulatory roles without entering passwords. In production, this will be replaced with AICTE's SSO / OpenID Connect (OIDC) identity provider.

Checklist

- [x] `POST /api/v1/auth/token` returns `200 OK`
- [x] Token format matches `dev-token-{userId}-{hash}`
- [x] LocalStorage key `aicte_auth_token` is populated
- [x] `GET /api/v1/auth/me` confirms server recognizes token
- **Result:** `PASS` `[ ] Fail [ ] Blocked`

---

5. Test Case 2: Applications Explorer List Retrieval

Test ID: TC-02

- **Feature Name:** Application List Retrieval & Backend Pagination Mapping
- **Purpose:** Verify that the Applications page retrieves application dossiers from PostgreSQL through Spring Boot, correctly maps pagination and fields, and allows client-side filtering.
- **Preconditions:** Logged in as Scrutiny Officer (`B. Singh`).
- **Primary Endpoint:** `GET /api/v1/applications?size=100`
- **Database Tables:** `applications` (joined with `institutions`)

Step-by-Step Instructions

1. In the left navigation sidebar, click **Application Explorer** (or navigate to `http://localhost:5173/applications`).

2. Inspect the Network tab in Developer Tools:

- Filter by `Fetch/XHR`.

- Locate the request: `GET /api/v1/applications?size=100`.

- Check Request Headers:

`text

Authorization: Bearer dev-token-USR-SO-01-xxxxxx

Content-Type: application/json

`

- Check Response Status: `200 OK`.

- Inspect Response JSON structure:

`json

{

"content": [

{

"id": 13,

"applicationReference": "APP-2026-1042",

"applicationType": "Extension of Approval / Compliance Review",

"academicYear": "2026-2027",

"status": "In progress",

"currentStage": "Scrutiny",

"ownerName": "B. Singh (Scrutiny Officer)",

"institution": {

"id": 17,

"institutionalIdentifier": "INST-1025",

"name": "ABC Institute of Technology",

"region": "Northern Region"

}

}

],

"page": 0,

"size": 100,

"totalElements": 2,

"totalPages": 1

}

`

3. Verify On-Screen Display:

- Row with Reference `APP-2026-1042` is prominently displayed.

- Institution Name displays **"ABC Institute of Technology"**.

- Region displays **"Northern Region"**.

- Current Stage badge shows **"Scrutiny"** (or current DB stage).

4. Test Filtering:

- Use the Search bar: type `1042`. Confirm only `APP-2026-1042` is shown.

- Clear search bar.

- Filter by Status dropdown: select `In progress`. Confirm list filters dynamically.

Expected Results

- Application records correspond to real PostgreSQL database entries.
- Backend DTO (`content`, `totalElements`) is cleanly mapped to frontend items without UI errors.

Checklist

- [x] Network tab displays `GET /api/v1/applications?size=100` returning `200 OK`
- [x] Record `APP-2026-1042` matches PostgreSQL row in `applications` table
- [x] Stage, owner, and institution name correspond to database values
- **Result:** `PASS` `[ ] Fail [ ] Blocked`

---

6. Test Case 3: Application Dossier (Detail View)

Test ID: TC-03

- **Feature Name:** Application Detail Retrieval & State Separation
- **Purpose:** Open the full dossier of `APP-2026-1042`, verify the live data retrieved from PostgreSQL, and distinguish between database-backed fields and frontend prototype presentations.
- **Preconditions:** Applications Explorer page open; logged in as Scrutiny Officer.
- **Direct Route:** `http://localhost:5173/applications/APP-2026-1042`
- **Backend Endpoint:** `GET /api/v1/applications/APP-2026-1042`

Step-by-Step Instructions

1. On the Applications Explorer list, click on row `APP-2026-1042` (or click the **Eye icon** action button).

2. The browser navigates to `/applications/APP-2026-1042`.

3. Verify the Header and Metadata Cards:

- **Application Reference:** `APP-2026-1042`

- **Institution:** `ABC Institute of Technology (INST-1025)`

- **Region:** `Northern Region`

- **Current Stage:** `Scrutiny` (or `Re-scrutiny` / `EVC scheduling`)

- **Assigned Owner:** `B. Singh (Scrutiny Officer)`

4. Inspect the **Dossier Navigation Tabs**:

- Click **Workflow Timeline** tab.

- Confirm timeline events show the real transition history.

- Look for the official timestamp, actor name, and deficiency remarks.

5. Identify Data Separation of Concerns:

- **Database-Backed Fields (Real PostgreSQL):**

- Application Reference (`application_reference`)

- Stage (`current_stage`)

- Status (`status`)

- Owner (`owner_name`)

- Due Date (`due_date`)

- Observations (`observations`)

- Deficiencies List (`deficiencies_json`)

- Optimistic Locking Version (`version`)

- Transition Events (`workflow_transitions`)

- **Frontend-Only Mock Elements:**

- Submissions file list (shows simulated PDF download links)

- Documentary checklist toggle items (saved locally in browser session)

Checklist

- [x] URL `/applications/APP-2026-1042` loads dossier cleanly
- [x] Header cards display exact values from PostgreSQL
- [x] Timeline accurately reflects database transitions
- **Result:** `PASS` `[ ] Fail [ ] Blocked`

---

7. Test Case 4: Workflow Action — Return for Correction

Test ID: TC-04

- **Feature Name:** Return Application for Deficiency Correction
- **Purpose:** Execute the Scrutiny Officer's action to issue a deficiency notice. Verify backend validation, atomic database updates, deficiency persistence, and audit logging.
- **Preconditions:**

- Logged in as **Scrutiny Officer** (`B. Singh`).

- Application `APP-2026-1042` is in stage `Scrutiny` or `Re-scrutiny` .

(If already completed, run the recovery SQL in Section 11 before testing).

- **Primary Endpoint:** `POST /api/v1/applications/APP-2026-1042/actions`
- **Database Tables Affected:** `applications` , `workflow_transitions` , `audit_events`

Step-by-Step Instructions

1. Open `/applications/APP-2026-1042` as Scrutiny Officer.

2. In the right-hand panel (or bottom action bar), locate the **Officer Action Controls**.

3. Click the red button: `Return for Correction` (with `CornerUpLeft` icon).

4. A modal dialog titled **"Return Application for Correction"** opens.

5. In the **Mandatory Return Reason** field, enter:

`text

Fire safety NOC expired. Certified faculty cadre roster missing required signatures.

`

6. In the **Deficiencies / Rectifications Required** field, enter each deficiency on a new line:

`text

Fire Safety NOC issued by Delhi Fire Service is expired (validity ended 2026-09-30)

Faculty cadre ratio roster missing HoD counter-signature for Computer Engineering

`

7. Click the red button: **`Confirm Return for Correction`**.

8. Inspect Developer Tools:

- **Network Request:** `POST /api/v1/applications/APP-2026-1042/actions`

- **Payload Sent:**

`json

{

"actionType": "RETURN_FOR_CORRECTION",

"actorName": "B. Singh",

"actorRole": "Scrutiny Officer",

"reason": "Fire safety NOC expired. Certified faculty cadre roster missing required signatures.",

"newDeficiencies": [

"Fire Safety NOC issued by Delhi Fire Service is expired (validity ended 2026-09-30)",

"Faculty cadre ratio roster missing HoD counter-signature for Computer Engineering"

],

"observations": "Land registration verified. Examining faculty cadre appointments."

}

`

- **HTTP Status:** `200 OK`

- **Response Payload:** Contains `"status": "Returned"`, `"currentStage": "Re-scrutiny"`, `"ownerName": "Institution Representative"`, and `"version": 1`.

9. Verify UI Changes:

- Modal closes automatically.

- Status badge changes from green/blue to amber/red **`Returned`**.

- Stage changes to **`Re-scrutiny`**.

- Owner changes to **`Institution Representative`**.

- Deficiencies card lists both deficiency bullet points.

- Workflow Timeline tab prepends a new event: *"Deficiency Notice Issued · Returned for Correction"*.

PostgreSQL Verification Query

Run in terminal:

```
PGPASSWORD=aicte_123 psql -h localhost -U aicte_app_user -d aicte_platform_db -c "
SELECT application_reference, current_stage, status, owner_name, version
FROM applications WHERE application_reference = 'APP-2026-1042';

SELECT action, from_stage, to_stage, from_status, to_status, actor_name, created_at
FROM workflow_transitions WHERE application_id = 13 ORDER BY id DESC LIMIT 1;

SELECT event_type, actor_id, target_entity_id, created_at
FROM audit_events WHERE target_entity_id = '13' ORDER BY id DESC LIMIT 1;
"
```

Expected SQL Output:

- `status` : `Returned`
- `current_stage` : `Re-scrutiny`
- `owner_name` : `Institution Representative`
- `action` : `RETURN_FOR_CORRECTION`
- `event_type` : `WORKFLOW_ACTION_RETURN_FOR_CORRECTION`

Checklist

- [x] Modal validates mandatory reason
- [x] `POST .../actions` returns `200 OK`
- [x] UI status updates to `Returned`
- [x] Deficiencies list persists to PostgreSQL `deficiencies_json`
- [x] `workflow_transitions` record created
- [x] `audit_events` record created with Jackson-escaped metadata
- **Result:** `PASS` `[ ] Fail [ ] Blocked`

---

8. Test Case 5: Workflow Action — Submit Corrections

Test ID: TC-05

- **Feature Name:** Submit Deficiency Rectification Dossier
- **Purpose:** Switch to the Institution Representative role and submit rectifications for a returned application. Verify backend validation requires `status == 'Returned'`.
- **Preconditions:**

- Application `APP-2026-1042` has status `Returned` (from TC-04).

- Switched to role **Institution Representative** (`Dr. Ramesh`).

- **Primary Endpoint:** `POST /api/v1/applications/APP-2026-1042/actions`
- **Database Tables Affected:** `applications` , `workflow_transitions` , `audit_events`

Step-by-Step Instructions

1. Click the top-right **Role Switcher** dropdown.

2. Select **Institution Representative** (`Dr. Ramesh (Principal)`).

- Network tab shows `POST /api/v1/auth/token` issuing `dev-token-USR-INST-01-...`.

3. Open the dossier: `/applications/APP-2026-1042`.

4. Observe the prominent yellow banner:

> **Deficiency Notice Issued — Institutional Rectification Required**

> The scrutiny desk has returned this dossier. Please rectify the listed items and resubmit.

5. In the banner or sidebar, click **`Submit Deficiency Rectification`** (or **`Resubmit Corrections`**).

6. A modal dialog titled **"Submit Rectifications"** opens.

7. In the **Explanatory Notes / Rectifications Summary** field, enter:

```text
Renewed Fire Safety NOC DFS/2026/884 attached. Certified faculty roster countersigned by HoD and Dean uploaded.
```

8. *(Optional UI simulation: file upload input is visible; note that only the textual explanation is persisted to PostgreSQL).*

9. Click **`Submit Rectifications for Re-scrutiny`**.

10. Inspect Developer Tools:

- **Network Request:** `POST /api/v1/applications/APP-2026-1042/actions`

- **Payload Sent:**

```json
{
"actionType": "SUBMIT_CORRECTIONS",
"actorName": "Dr. Ramesh (Principal)",
"actorRole": "Institution Representative",
"reason": "Renewed Fire Safety NOC DFS/2026/884 attached. Certified faculty roster countersigned by HoD and Dean uploaded."
}
```

- **HTTP Status:** `200 OK`

- **Response Payload:** Contains `"status": "In progress"`, `"currentStage": "Re-scrutiny"`, `"ownerName": "B. Singh (Scrutiny Officer)"`, `"deficiencies": []`, and `"version": 2`.

11. Verify UI Changes:

- Modal closes.

- Yellow deficiency warning banner disappears.

- Status badge updates to **`In progress`**.

- Assigned owner shifts back to **`B. Singh (Scrutiny Officer)`**.

- Timeline tab appends event: *"Corrections Resubmitted by Institution"*.

PostgreSQL Verification Query

```
PGPASSWORD=aicte_123 psql -h localhost -U aicte_app_user -d aicte_platform_db -c "
SELECT application_reference, current_stage, status, owner_name, version
FROM applications WHERE application_reference = 'APP-2026-1042';

SELECT action, from_stage, to_stage, from_status, to_status, actor_name
FROM workflow_transitions WHERE application_id = 13 ORDER BY id DESC LIMIT 1;

SELECT event_type, actor_id FROM audit_events WHERE target_entity_id = '13' ORDER BY id DESC LIMIT 1;
"
```

Expected SQL Output:

- `status` : `In progress`
- `current_stage` : `Re-scrutiny`
- `owner_name` : `B. Singh (Scrutiny Officer)`
- `action` : `SUBMIT_CORRECTIONS`
- `event_type` : `WORKFLOW_ACTION_SUBMIT_CORRECTIONS`

Checklist

- [x] Only available when application status is `Returned`
- [x] Network tab verifies `POST .../actions` with `SUBMIT_CORRECTIONS`
- [x] Application reassigns to Scrutiny Officer in PostgreSQL
- [x] Deficiencies list cleared pending re-evaluation
- [x] Audit event and transition record persisted
- **Result:** `PASS` `[ ] Fail [ ] Blocked`

---

9. Test Case 6: Workflow Action — Complete Scrutiny

Test ID: TC-06

- **Feature Name:** Clear Scrutiny & Advance to Next Stage
- **Purpose:** Switch back to Scrutiny Officer and complete the scrutiny evaluation. Verify application advances to `EVC scheduling`, updates ownership, and records audit trail.
- **Preconditions:**

- Application `APP-2026-1042` is in stage `Re-scrutiny`, status `In progress` (from TC-05).

- Switched back to role **Scrutiny Officer** (`B. Singh`).

- **Primary Endpoint:** `POST /api/v1/applications/APP-2026-1042/actions`
- **Database Tables Affected:** `applications` , `workflow_transitions` , `audit_events`

Step-by-Step Instructions

1. Switch role back to **Scrutiny Officer** (`B. Singh`) using the top-right dropdown.

2. Open `/applications/APP-2026-1042` .

3. In the Officer Action panel, click the green button: **`Complete Scrutiny`** (with `CheckCircle2` icon).

4. A modal titled **"Complete Scrutiny Evaluation"** appears.

5. In the **Forward Application To Next Stage** dropdown, select: **`EVC scheduling`** (or `Regional Committee review`).

6. In the **Desk Remarks / Final Recommendation** field, enter:

`text

All fire safety and faculty roster deficiencies rectified and verified. Cleared for on-site EVC scheduling.

`

7. Click **`Confirm & Complete Scrutiny`**.

8. Inspect Developer Tools:

- **Network Request:** `POST /api/v1/applications/APP-2026-1042/actions`

- **Payload Sent:**

`json

{

"actionType": "COMPLETE_SCRUTINY",

"actorName": "B. Singh",

"actorRole": "Scrutiny Officer",

"nextStage": "EVC scheduling",

"observations": "All fire safety and faculty roster deficiencies rectified and verified. Cleared for on-site EVC scheduling."

}

`

- **HTTP Status:** `200 OK`

- **Response Payload:** Contains `"status": "Due soon"`, `"currentStage": "EVC scheduling"`, `"ownerName": "C. Verma (EVC Coordinator)"`, and `"version": 3`.

9. Verify UI Changes:

- Modal closes.

- Stage badge updates to **`EVC scheduling`**.

- Status badge updates to **`Due soon`**.

- Assigned owner updates to **`C. Verma (EVC Coordinator)`**.

- Workflow Timeline displays the complete 3-event transition chain.

PostgreSQL Verification Query

```
PGPASSWORD=aicte_123 psql -h localhost -U aicte_app_user -d aicte_platform_db -c "
SELECT application_reference, current_stage, status, owner_name, version, updated_at
FROM applications WHERE application_reference = 'APP-2026-1042';

SELECT id, action, from_stage, to_stage, from_status, to_status, actor_name
FROM workflow_transitions WHERE application_id = 13 ORDER BY id ASC;
"
```

Expected SQL Output:

- `current_stage`: `EVC scheduling`
- `status`: `Due soon`
- `owner_name`: `C. Verma (EVC Coordinator)`

- `version` : `3`
- Exactly 3 rows in `workflow_transitions` (`RETURN_FOR_CORRECTION` -> `SUBMIT_CORRECTIONS` -> `COMPLETE_SCRUTINY`).

Checklist

- [x] Application advances to `EVC scheduling`
- [x] Ownership shifts to EVC Coordinator
- [x] Optimistic lock version increments to `3`
- [x] Chronological timeline in UI matches PostgreSQL records
- **Result:** `PASS` `[ ] Fail [ ] Blocked`

---

10. Test Case 7: Negative Testing — Invalid State Transition Guard

Test ID: TC-07

- **Feature Name:** State Machine Prerequisite Enforcement & RFC 7807 Rejection
- **Purpose:** Verify that attempting an illegal workflow transition (such as attempting to complete scrutiny on an application already in `EVC scheduling`) is rejected by the backend with an RFC 7807 Problem Detail error and does not corrupt database state.
- **Preconditions:** Application `APP-2026-1042` is in stage `EVC scheduling` (from TC-06).
- **Endpoint Tested:** `POST /api/v1/applications/APP-2026-1042/actions`

Step-by-Step Instructions

1. As Scrutiny Officer, while viewing `APP-2026-1042` (now in `EVC scheduling`), attempt to trigger a scrutiny completion again, or execute the equivalent HTTP command via terminal:

`bash

TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/token -H "Content-Type: application/json" -d '{"role":"Scrutiny Officer"}' | python3 -c "import sys, json; print(json.load(sys.stdin)['access_token'])")

curl -i -X POST "http://localhost:8080/api/v1/applications/APP-2026-1042/actions" -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{

"actionType": "COMPLETE_SCRUTINY",

"actorName": "B. Singh",

"actorRole": "Scrutiny Officer"

}'

`

2. Observe the HTTP Response:

- **HTTP Status:** `400 Bad Request`

- **Content-Type:** `application/problem+json`

- **Problem Detail Response Body:**

`json

{

"type": "https://api.aicte.gov.in/errors/bad-request",

"title": "Invalid Request Parameter",

"status": 400,

"detail": "Cannot clear scrutiny from stage 'EVC scheduling'. Must be in Scrutiny or Re-scrutiny.",

"instance": "/api/v1/applications/APP-2026-1042/actions"

}

`

3. In the UI (if triggered via browser):

- The UI displays an alert containing the exact backend error message.

- The application does not transition and local state is not updated.

4. Verify Database Integrity:

- Run the verification query to confirm `version` remains `3` and no additional transition record was created.

Checklist

- [x] Backend rejects invalid stage transition with HTTP `400 Bad Request`
- [x] RFC 7807 `application/problem+json` response contains actionable `detail`
- [x] Database state remains uncorrupted
- **Result:** `PASS` `[ ] Fail [ ] Blocked`

---

11. Test Case 8: Unified End-to-End Rehearsal & Safe Recovery

Test ID: TC-08

- **Feature Name:** Unified Scrutiny & Deficiency-Rectification Journey
- **Purpose:** Execute the entire demonstration flow sequentially from start to finish.

Journey Sequence Summary

```
1. Login as Scrutiny Officer
      |
2. Open Applications Explorer (/applications)
      |
3. Open Dossier APP-2026-1042 (Initial Stage: Scrutiny)
      |
4. Return for Correction (Enters 2 deficiencies)
   ──► State: Re-scrutiny | Status: Returned | Owner: Institution Rep
      |
5. Switch Role to Institution Representative
      |
6. Submit Rectification Dossier (Enters rectification notes)
   ──► State: Re-scrutiny | Status: In progress | Owner: Scrutiny Officer
      |
7. Switch Role back to Scrutiny Officer
      |
8. Complete Scrutiny (Forward to EVC scheduling)
   ──► State: EVC scheduling | Status: Due soon | Owner: EVC Coordinator
      |
9. Verify PostgreSQL 18:
   - 3 records in workflow_transitions
   - 3 records in audit_events
   - Version = 3 in applications
```

Safe Demonstration Reset Query (Non-Destructive)

If you need to re-run the demonstration on `APP-2026-1042` without clearing existing history or dropping tables, execute this safe SQL update:

```
-- Reset application to initial Scrutiny state for re-demonstration
UPDATE applications
SET current_stage = 'Scrutiny',
    status = 'In progress',
    owner_name = 'B. Singh (Scrutiny Officer)',
    owner_id = 'USR-SO-01',
    deficiencies_json = '[]',
    observations = 'Land registration verified. Examining faculty cadre appointments.'
WHERE application_reference = 'APP-2026-1042';
```

11b. Test Case 9: Create Application Workflow (Institution Representative)

Test ID: TC-09

- **Feature Name:** Create New Application Dossier
- **Purpose:** Verify that an Institution Representative can click "New Application", fill in canonical application fields (Institution, Reference, Type, Academic Year), submit directly to `POST /api/v1/applications`, receive server confirmation, refresh the live list, and open the new dossier.
- **Preconditions:** Logged in as **Institution Representative** (`Dr. Ramesh Kumar`).
- **Primary Endpoint:** `POST /api/v1/applications`
- **Lookup Endpoint:** `GET /api/v1/institutions`
- **Database Tables Affected:** `applications` , `audit_events`

Step-by-Step Instructions

1. Log in or switch role to **Institution Representative** (`Dr. Ramesh Kumar`).

2. Navigate to **Application Explorer** (`/applications`).

3. Locate the blue **`New Application`** button (with `+` icon, `id="btnNewApplication"`) positioned in the header toolbar next to the Scope badge.

(Note: This button is strictly hidden for Scrutiny, Regional, and other bureau roles).

4. Click **`New Application`** .

5. The **"Create New Application Dossier"** modal dialog opens:

- The **Canonical Institution** dropdown automatically queries `GET /api/v1/institutions` and pre-selects the user's institution: `ABC Institute of Technology (INST-1025)` (value: 17).

- The **Application Reference Number** field is pre-filled with a suggested unique identifier (e.g. `APP-2026-XXXX`).

- Select the **Application Type** (e.g. `Extension of Approval (EoA)`).

- Select the **Academic Approval Year** (e.g. `2026-2027`).

6. Click **`Register Application`** (`id="btnSubmitCreateApplication"`).

7. Inspect Developer Tools Network Tab:

- **Request URL:** `POST http://localhost:8080/api/v1/applications`

- **Request Headers:** `Authorization: Bearer dev-token-USR-INST-01-...`

- **Payload Sent:**

`json

{

"institutionId": 17,

"applicationReference": "APP-2026-XXXX",

"applicationType": "Extension of Approval (EoA)",

"academicYear": "2026-2027"

}

`

- **Response Status:** `201 Created`

- **Response Body:** Full `ApplicationResponse` with status `DRAFT`, stage `Scrutiny`, version `0`.

8. Verify UI Result:

- Modal displays the green **"Application Registered Successfully!"** confirmation panel.

- Click `Open Application Dossier` (`id="btnViewCreatedDossier"`).

- The browser navigates directly to `/applications/APP-2026-XXXX`.

- The newly created dossier is loaded directly from PostgreSQL!

Negative Testing Scenarios Verified

1. **Invalid / Blank Input:** Leaving required fields blank triggers an RFC 7807 `400 Bad Request` Problem Detail response with field errors displayed under each input.

2. **Duplicate Application Reference:** Submitting an existing reference triggers an RFC 7807 `409 Conflict` response (`detail: Application already exists with reference: ...`).

3. **Unauthorized Request:** Submitting without a valid Bearer token returns HTTP `401 Unauthorized`.

PostgreSQL Verification Query

```
PGPASSWORD=aicte_123 psql -h localhost -U aicte_app_user -d aicte_platform_db -c "
SELECT id, application_reference, current_stage, status, version, created_at
FROM applications
WHERE application_reference = 'APP-2026-XXXX';

SELECT id, event_type, actor_id, target_entity_id, metadata_json
FROM audit_events
WHERE target_entity_id = (SELECT id::text FROM applications WHERE application_reference = 'APP-2026-XXX
"
```

Checklist

- [x] "New Application" button visible only to Institution Representatives
- [x] Form pre-populates canonical institution ID from `GET /api/v1/institutions`
- [x] Submits directly to `POST /api/v1/applications` with Bearer token
- [x] Returns `201 Created` and persists to PostgreSQL `applications` & `audit_events`
- [x] Refreshes list from backend API
- [x] Allows opening the newly created application dossier

- **Result:** `PASS` `[ ] Fail [ ] Blocked`

12. Non-Connected UI Features Investigation

As required by the assessment mandate, we investigated whether Application Creation and Institution Management are exposed as backend-connected UI features:

12.1 Application Creation (`POST /api/v1/applications`)

- **Backend Capability:** A robust REST endpoint `POST /api/v1/applications` exists, accepting `institutionId`, `applicationReference`, `applicationType`, and `academicYear`. It validates inputs, ensures uniqueness, and logs audit events.
- **Frontend Integration: Implemented & Fully Connected!** Institution Representatives can create new applications via the "New Application" modal on the Applications Explorer page (`ApplicationsPage.jsx`). See **Test Case TC-09** in Section 11b.

12.2 Institution Management (`/api/v1/institutions`)

- **Backend Capability:** Full CRUD endpoints exist (`GET /api/v1/institutions` , `POST /api/v1/institutions` , `GET /api/v1/institutions/{id}`).
- **Frontend Code Inspection:** The frontend uses static metadata from `mockInstitutions.js` within dashboard selectors. There is no Institution list, registration form, or management interface in the React router calling `/api/v1/institutions` .
- **Conclusion: Institution management is NOT exposed in the current React frontend.**

---

13. Out of Scope — Frontend-Only Prototype Features

The following features remain safely isolated on client-side `localStorage` or in-memory mock datasets. **Do not attempt to verify them against PostgreSQL:**

1. **EVC Scheduling & Visit Allocation (`/evc`):**

- Managed via `aicte_evc_store` in browser `localStorage` .

- Inspection dates and expert panel assignments do not hit the backend API.

2. **Post-Approval Compliance Obligations (`/compliance`):**

- Managed via `aicte_compliance_store` .

- Undertaking evidence uploads and review statuses are stored in browser memory.

3. **Statutory Document Registry & Expiry Tracking (`/documents`):**

- Managed via `aicte_documents_store` .

- Document upload simulations and renewal status changes remain client-side.

4. **Alerts, Notifications & Escalations (`/alerts`):**

- Managed via `aicte_alerts_store` .

- Escalation countdowns and simulated delivery failures operate locally.

5. **BI Analytics & Executive Dashboards (`/reports`):**

- Powered by `dataService.getRoleKPIs()` calculating in-memory counts from localStorage keys.

6. **OCR & Document Integrity Checks:**

- Visual mock indicators representing simulated document checks.

---

14. Manual Rehearsal Verification Sign-off Table

Test Case	Feature Tested	Tester Name	Date Executed	Status (Pass / Fail)	Sign-off Notes
TC-01	Demo Login & Role Switching	_____	____ / ____ / 2026	`[ ] Pass [ ] Fail`	`POST /auth/token` verified
TC-02	Applications Explorer List	_____	____ / ____ / 2026	`[ ] Pass [ ] Fail`	`GET /applications` loaded from DB
TC-03	Application Dossier Loading	_____	____ / ____ / 2026	`[ ] Pass [ ] Fail`	`APP-2026-1042` matched DB row
TC-04	Return for Correction	_____	____ / ____ / 2026	`[ ] Pass [ ] Fail`	Transition & audit logged in DB
TC-05	Submit Corrections	_____	____ / ____ / 2026	`[ ] Pass [ ] Fail`	Reassigned to Scrutiny Officer
TC-06	Complete Scrutiny	_____	____ / ____ / 2026	`[ ] Pass [ ] Fail`	Advanced to `EVC scheduling`
TC-07	Invalid State Rejection	_____	____ / ____ / 2026	`[ ] Pass [ ] Fail`	RFC 7807 400 Problem Detail verified
TC-08	Full End-to-End Journey	_____	____ / ____ / 2026	`[ ] Pass [ ] Fail`	All PostgreSQL records verified

---

AICTE Approval & Compliance Orchestration Platform (SIH 26262) — Manual Testing Documentation